

INTELLIGENCE COMPORTEMENTALE

CLIENTÈLE RETAIL

RÉALISÉ PAR
EYA DAMAK
GI2S4

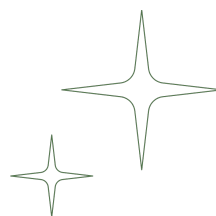
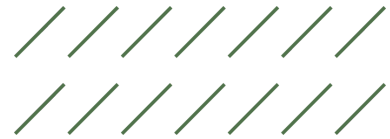




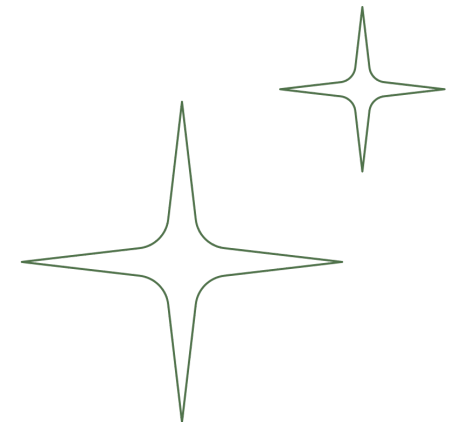
Table de matières

- Introduction
- Problématique
- Exploration des données
- Traitement des données
- ACP et Clustering K-means
- Modèle de classification
- Modèle de Régression
- Déploiement Flask
- Conclusion

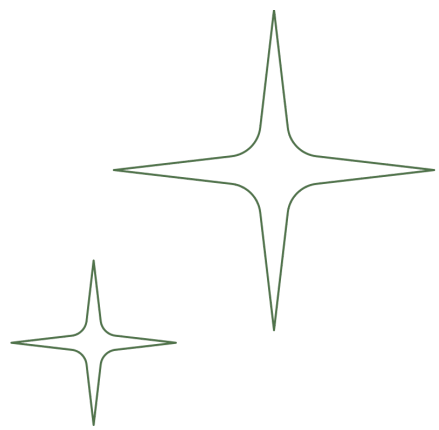


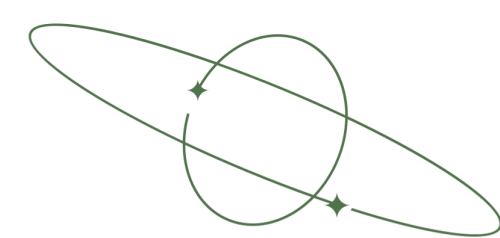


Introduction



- Le secteur du retail e-commerce génère des volumes massifs de données clients. Fidéliser un client existant coûte 5 à 7 fois moins cher qu'en acquérir un nouveau — anticiper le churn devient donc un enjeu stratégique majeur
- Ce projet applique des techniques de Machine Learning sur un dataset réel d'un e-commerce de cadeaux (4 372 clients, 52 variables) pour analyser les comportements clients et prédire leur risque de départ.





problématique

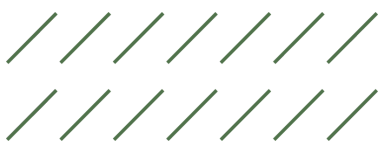
Prédiction du churn et valorisation client à partir des données comportementales d'un e-commerce
Trois sous-problèmes traités :

Segmentation —
Quels profils de clients
existe-t-il ? (ACP + K-
Means)

Classification —
Ce client va-t-il partir ?
(Random Forest,
XGBoost, Stacking)

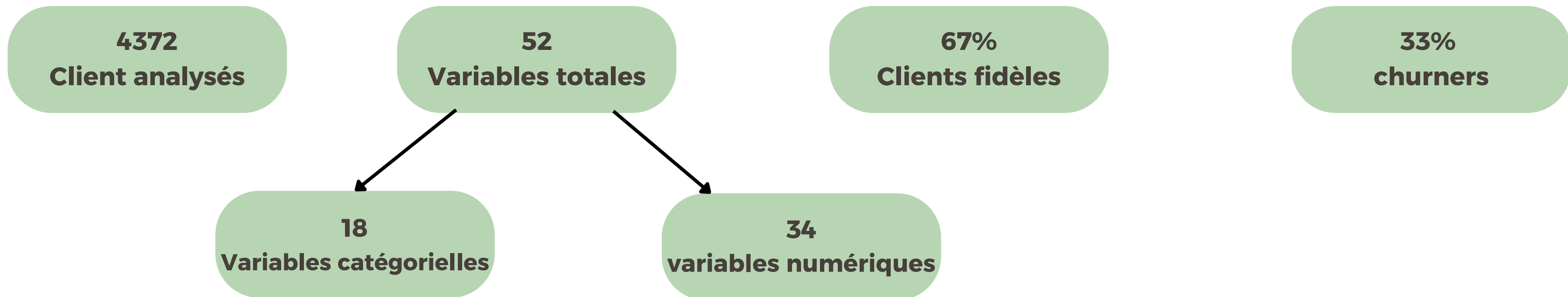
Régression —
Combien ce client va-t-
il dépenser ? (XGBoost
Regressor)

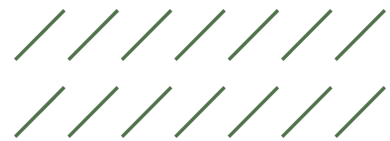
Garantir des modèles réellement prédictifs en évitant le data leakage — problème détecté et corrigé durant le projet (accuracy 100% → 94,9%).



Exploration des données

Objectif : Comprendre la structure et la qualité du dataset avant toute modélisation



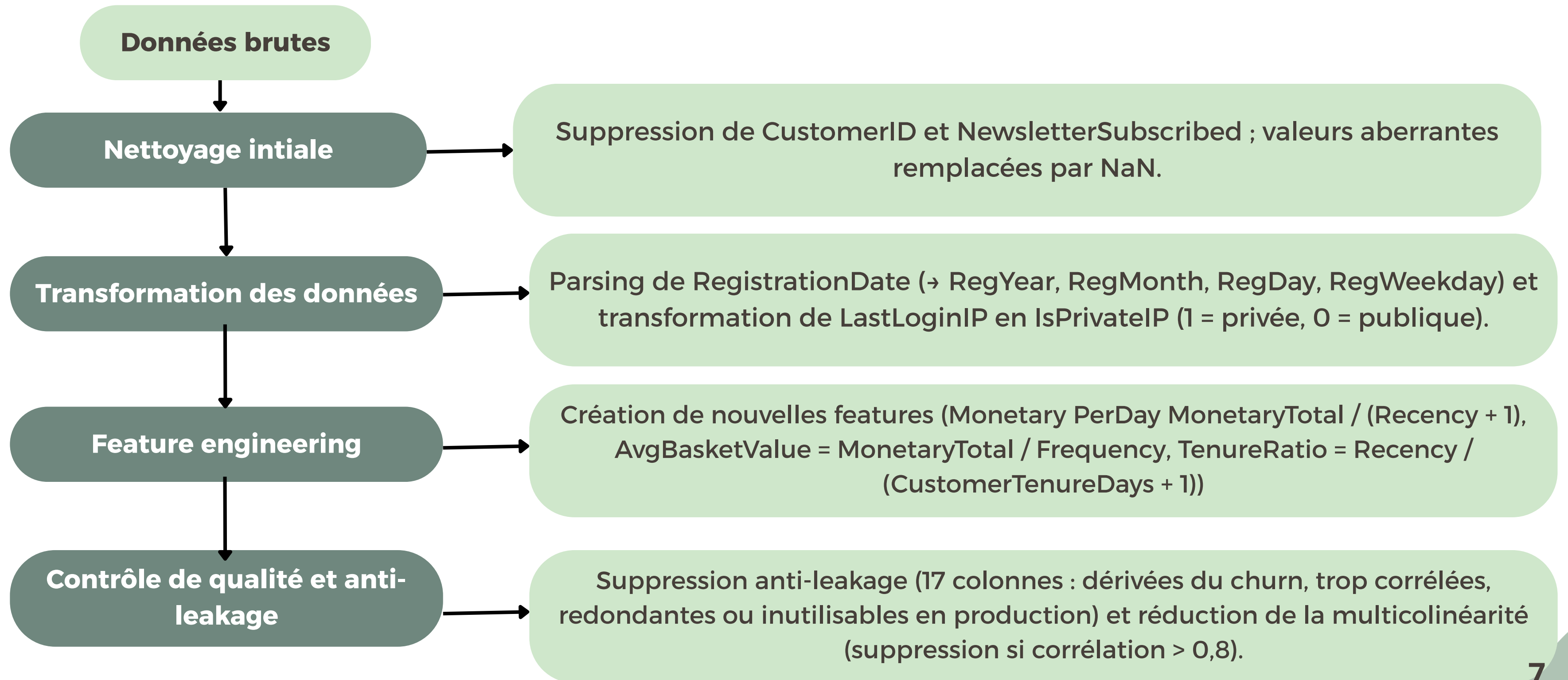


Exploration des données

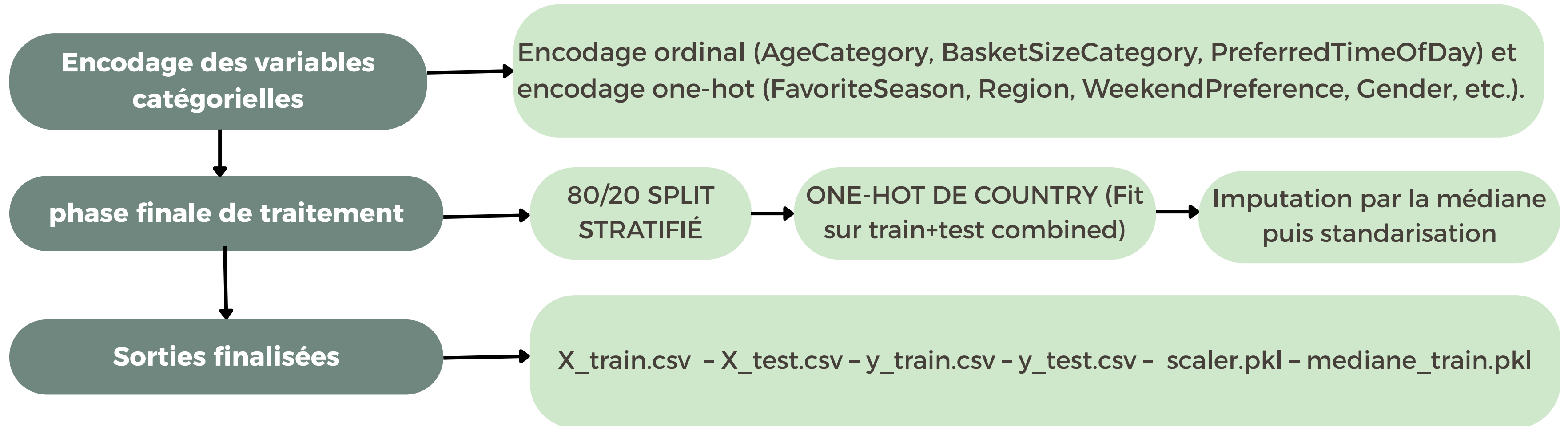
Étapes de l'exploration

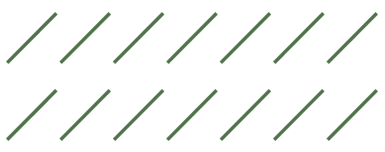
Étape	Outil utilisé	Le cible recherché	Résultat
Aperçu général	df.head() / df.info()	Structure du dataset	4 372 clients, 52 variables
Statistiques descriptives	df.describe()	Distributions des valeurs	Moyennes, écarts-types, quartiles
Valeurs manquantes	df.isnull().sum() + barplot	Colonnes incomplètes	Plusieurs colonnes avec NaN
Doublons	df.duplicated()	Lignes répétées	Aucun doublon détecté
Valeurs aberrantes	Boxplots + histogrammes	Valeurs hors-plage	Sentinelles 999 / 99 / -1 détectées
Variable cible	value_counts() + barplot	Déséquilibre des classes	Déséquilibre churners / fidèles
Corrélations	Heatmap sns.heatmap()	Features redondantes	Paires fortement corrélées (
Multicolinéarité	VIF (statsmodels)	Redondance entre features	Features avec VIF > 10 identifiées
Variables catégorielles	value_counts()	Distribution des modalités	3 formats dans RegistrationDate, variance nulle dans NewsletterSubscribed

Traitement des données

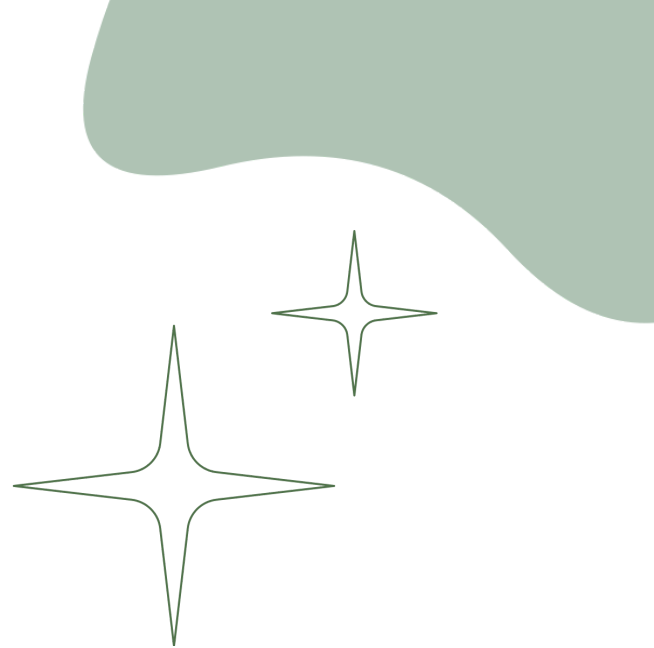


Traitement des données

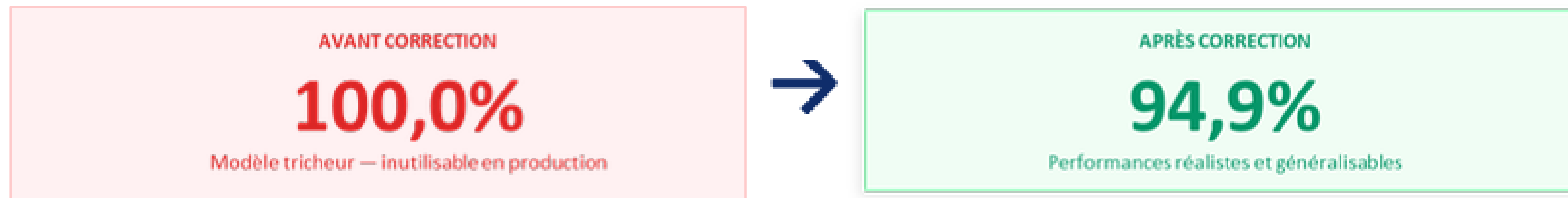


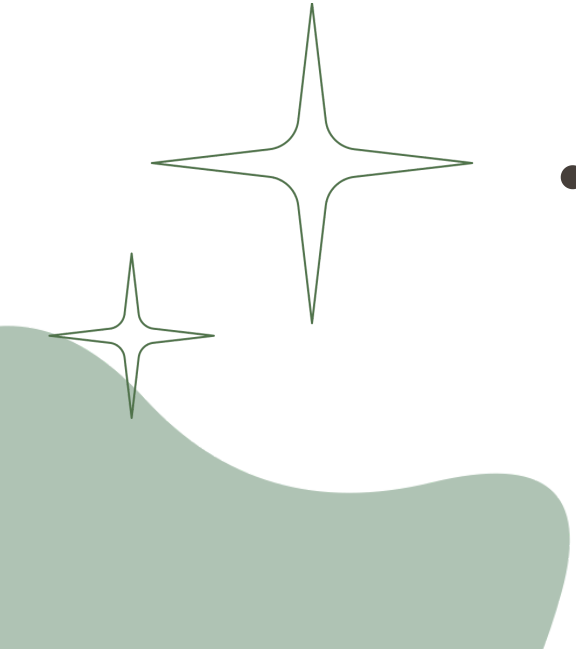


Data leakage



- Le data leakage se produit quand le modèle apprend à partir d'informations qu'il n'aurait jamais en production — il "triche" sans qu'on s'en rende compte.
- Impact sur le modèle random forest:



- 
- Suppression des variables dérivées du churn, trop corrélées ou redondantes afin d'éviter la fuite de données et les biais.
 - Total : 17 colonnes supprimées.
- 

ACP



POURQUOI L'ACP ?

Après tous les encodages One-Hot, le dataset atteint **87 features**

Problèmes en haute dimension :

- Malédiction de la dimensionnalité
- Temps de calcul accru
- Clusters de mauvaise qualité
- Risque de surapprentissage

RÉSULTAT CLÉ

56

composantes conservent 95,5% de la variance

31 dimensions supprimées = bruit + redondances

Modèle sauvegardé → models/pca.pkl

Sans ACP

0,038

Score de silhouette K-Means



Avec ACP

0,086

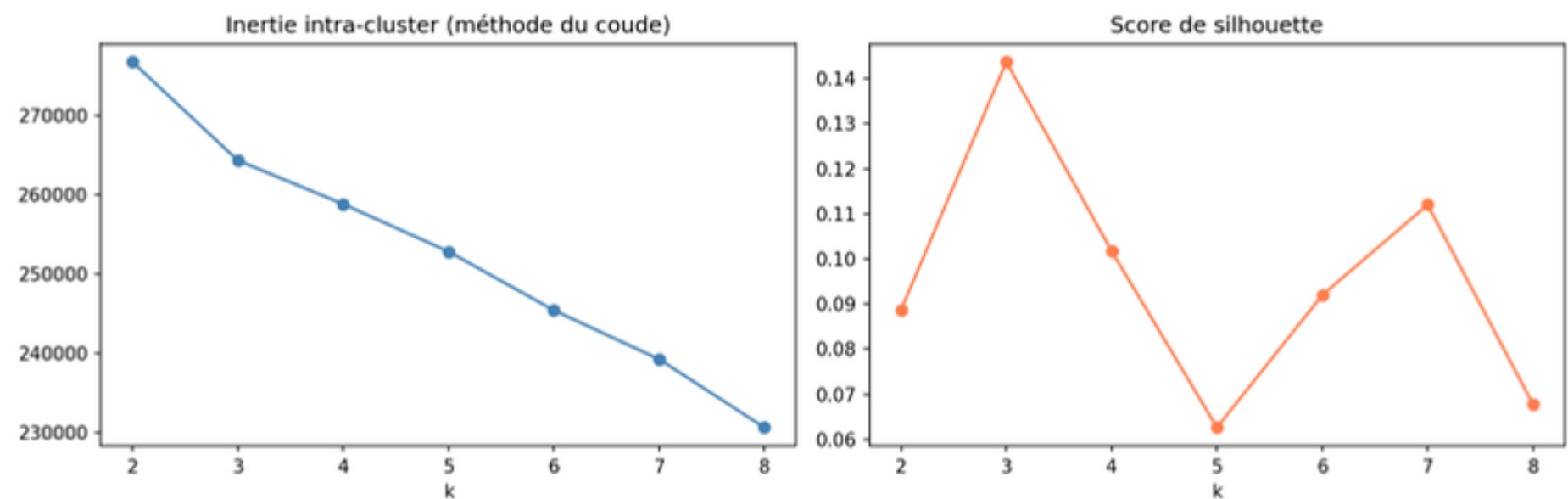
Score de silhouette K-Means



Clustering K-means



SÉLECTION DU k OPTIMAL : méthode du coude + score de silhouette → k = 4



Segment	Effectif	%	Churn (%)	Caractéristiques
Premium	935	26,70%	21,20%	Fréquence & montant élevés, churn le plus bas
Réguliers	2317	66,30%	38,40%	Fréquence & montant faibles, churn élevé
Occasionnels	7	0,20%	10,10%	Faible effectif
Inactifs	238	6,80%	30,30%	Clients peu actifs

Modèles de classification



Modèle Random Forest

Principe : Ensemble d'arbres de décision qui votent ensemble pour prédire si un client va churner ou non.

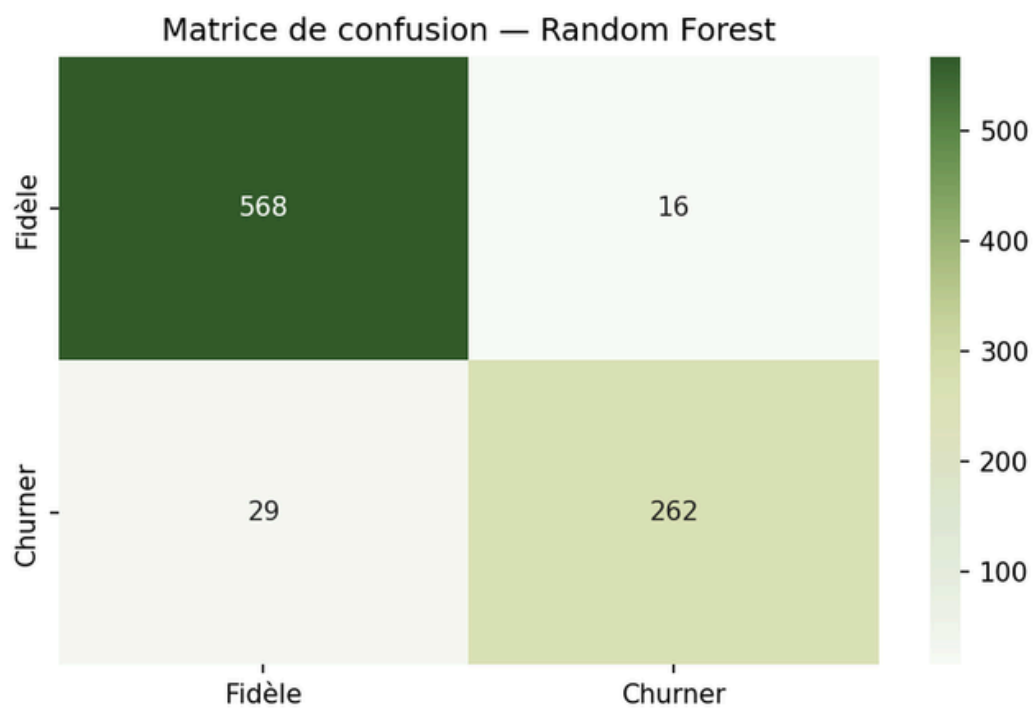
Étape	Détail
Anti-leakage	Corrélation > 0,8 avec Churn → suppression automatique
Optimisation	RandomizedSearchCV — 60 combinaisons, CV = 5
Seuil optimisé	Entre 0,25 et 0,75 pour maximiser le F1-score
Déséquilibre	class_weight = 'balanced'

PERFORMANCES (seuil = 0,46)

Accuracy	AUC
94,9 %	0,985
Précision churner	Seuil optimal
0,94	0,46

RandomizedSearchCV

Hyperparamètre	Valeur retenue
n_estimators	800 arbres
max_depth	20
min_samples_split	5
min_samples_leaf	2
max_features	0.7



Modèles de classification



Modèle XGboost

XGBoost Algorithme de boosting qui construit les arbres séquentiellement — chaque arbre corrige les erreurs du précédent

Gestion du déséquilibre — SMOTE

Avant l'entraînement, SMOTE génère des exemples synthétiques de churners :

Avant : 2334 fidèles / 778 churners

Après : 2334 fidèles / 1167 churners (ratio 0.5)

GridSearchCV

Hyperparamètre	Valeur retenue
n_estimators	500 arbres
max_depth	5
learning_rate	0.08
subsample	0.9
colsample_bytree	0.9
scale_pos_weight	3
Seuil optimal	0.62

PERFORMANCES (seuil = 0,46)

Accuracy

97 %

AUC

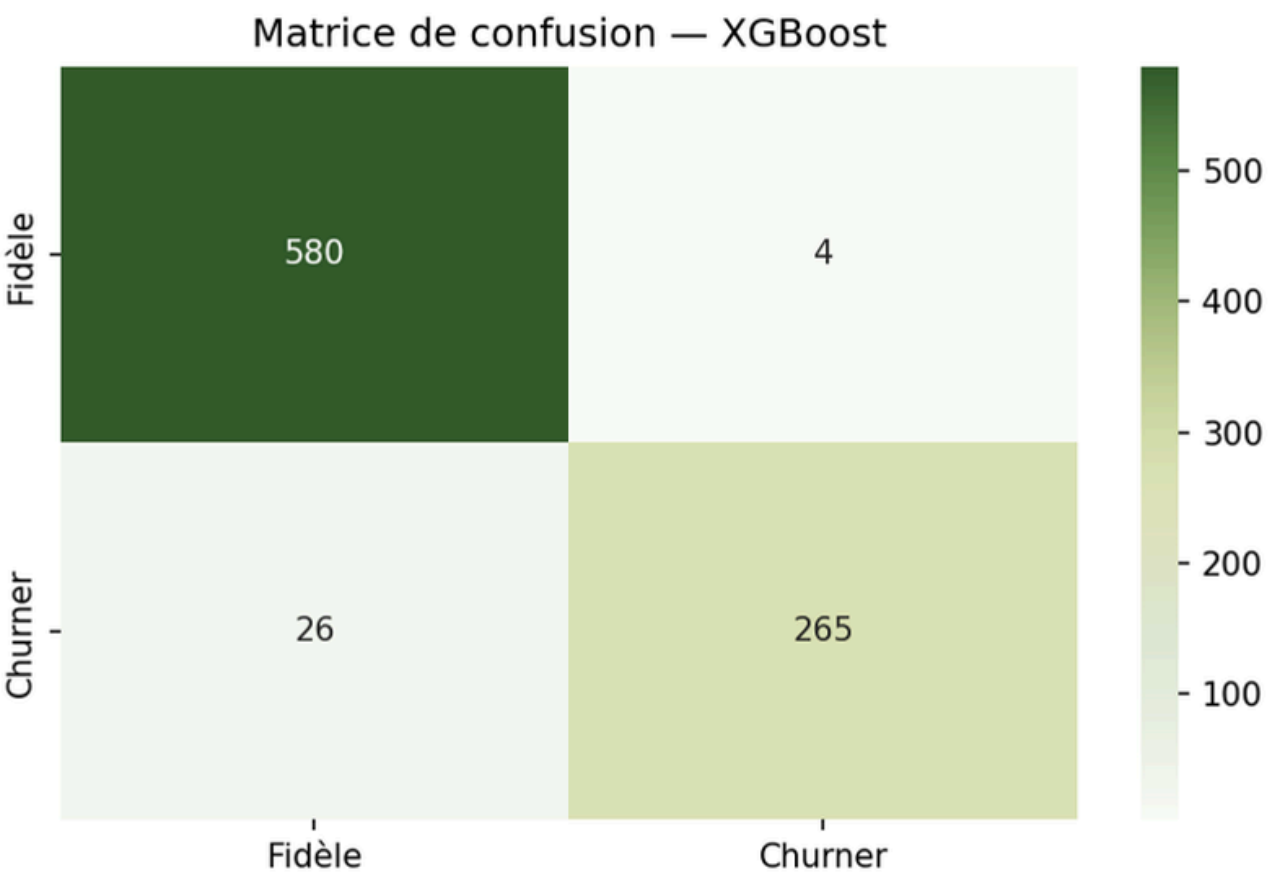
0,994

Précision churner

0,94

Seuil optimal

0,50



Modèles de classification



Modèle Stacking

Définition : Combine RF et XGBoost en utilisant leurs probabilités comme entrées d'un méta-modèle (Régression Logistique) qui prend la décision finale.

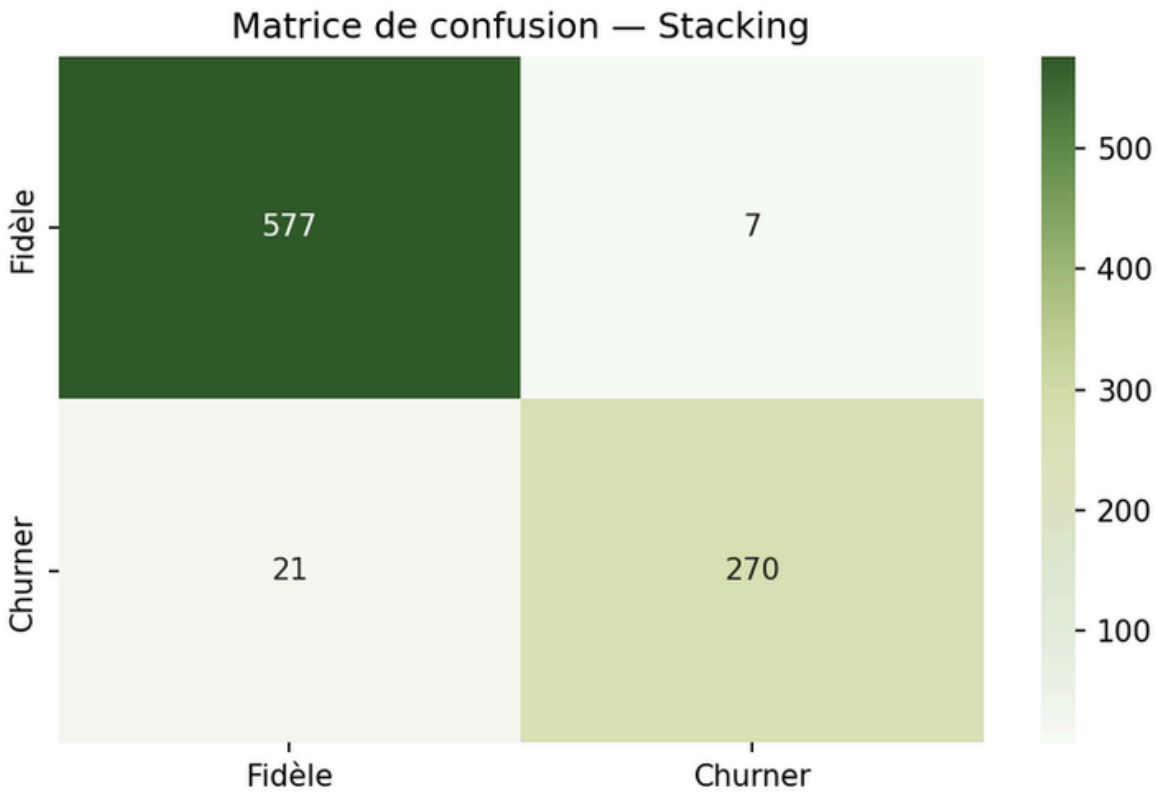
Niveau	Modèle	Rôle
Niveau 1	Random Forest	Prédiction de probabilités
Niveau 1	XGBoost	Prédiction de probabilités
Niveau 2	Régression Logistique	Décision finale

Modèle	Poids
Random Forest	3.32
XGBoost	4.98



Le méta-modèle fait plus confiance à XGBoost qu'au Random Forest.

PERFORMANCES	
Accuracy	AUC
96,8 %	0,992
Précision churner	Seuil optimal
0,97	0,50



Comparaison des Modèles &

Choix Final

Modèle	Accuracy	AUC	F1 Churner	Précision Churner	Rappel Churner
Random Forest	94.9%	0.985	91%	95%	88%
XGBoost	96.6%	0.994	95%	99%	91%
Stacking	96.8%	0.992	95%	97%	93%

XGBoost est le modèle retenu car il combine la meilleure accuracy (97%), le meilleur AUC (0.994) et la meilleure précision churner (99%) — tout en restant simple à déployer en production.

Modèles de Regression

Modèle XGBoost

Objectif : Prédire le montant total dépensé par un client (MonetaryTotal) en £.

RandomizedSearchCV

Hyperparamètre	Valeur
n_estimators	300
max_depth	6
learning_rate	0.01
subsample	0.7
colsample_bytree	0.7

PERFORMANCES

R² cross-validation

0.784

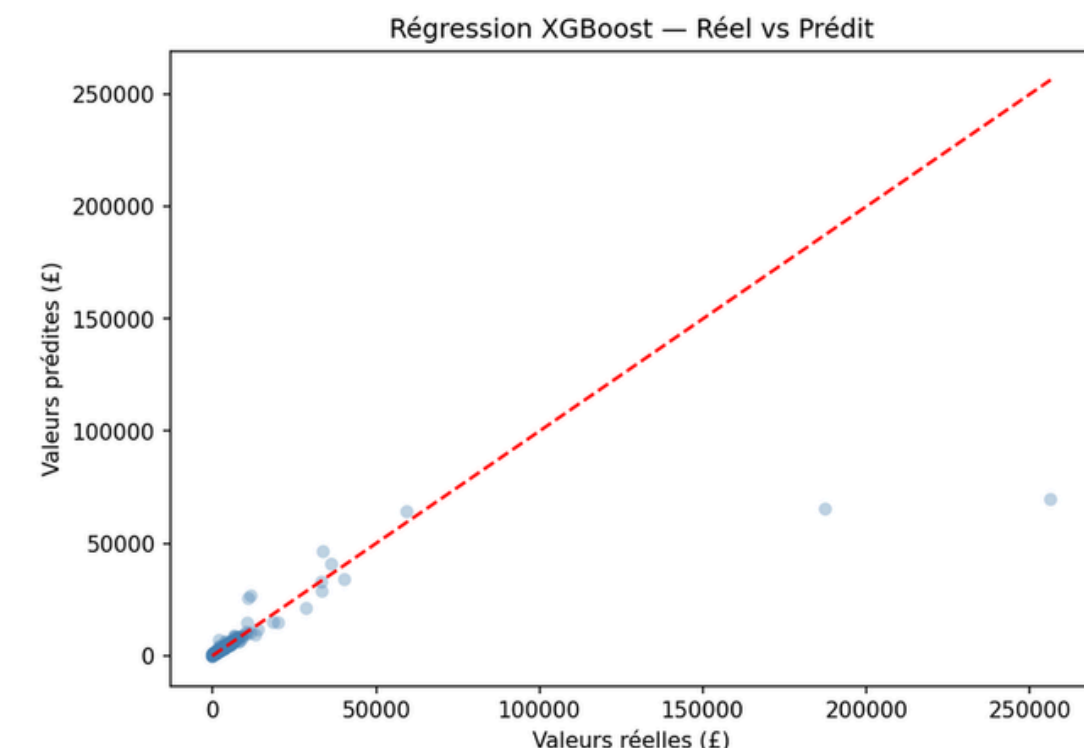
RMSE

7 621.71 £

R² test

0.549

- Le modèle explique 78% de la variance en cross-validation
- Sur données inconnues ce taux descend à 54.9% — MonetaryTotal est une variable difficile à prédire car influencée par des facteurs externes non capturés (promotions, saisonnalité)
- La transformation log1p a été appliquée sur la cible pour stabiliser les grandes valeurs

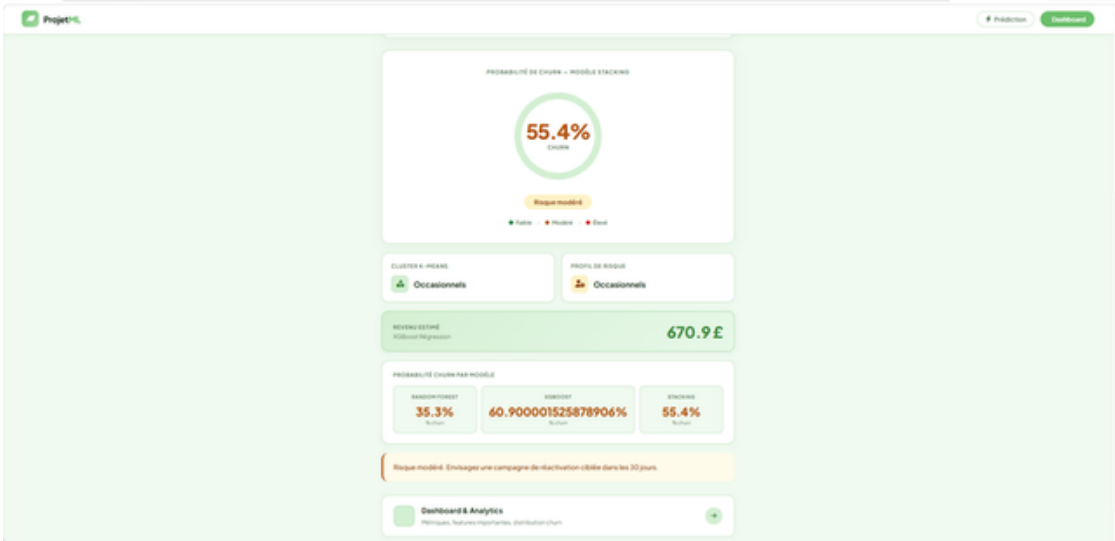


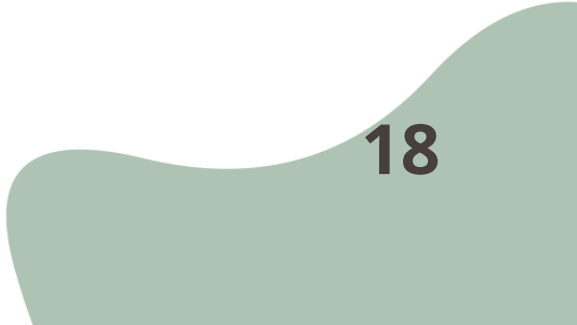
Déploiement flask



Composant	Détail
Serveur	Flask — app.py
Interface	index.html + dashboard.html
Modèles chargés	random_forest.pkl xgboost.pkl kmeans.pkl + pca.pkl stacking.pkl regression_xgboost_optimized.pkl scaler.pkl + scaler_regression.pkl

- **Inputs (4 variables):** Fréquence · Montant · Saison · Diversité produits
- **Outputs par client :**
Probabilité de churn (RF / XGBoost / Stacking)
Segment K-Means → Premium / Réguliers / Occasionnels / Inactifs
Revenu estimé en £





Conclusion

Ce projet a permis de construire un pipeline ML complet, de l'exploration au déploiement, avec XGBoost comme modèle final (Accuracy 97%, AUC 0.994). La correction du data leakage reste l'apport technique majeur. En perspective, un déploiement cloud et des alertes automatiques constitueraient les prochaines étapes.



Merci pour votre
attention!